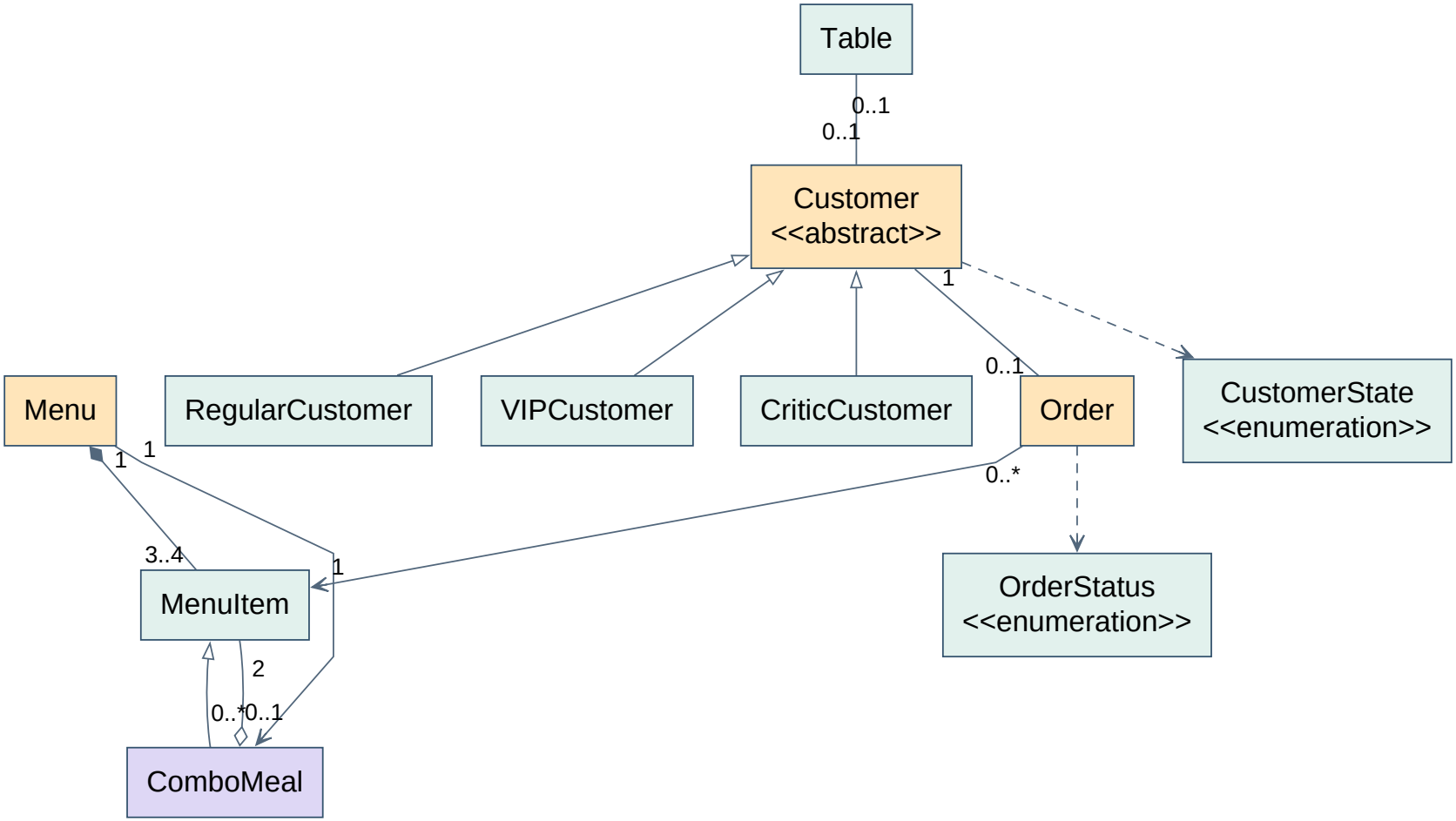
RESTAURANT RUSH / STAGE 3

Class relationships - customers, orders and menu

UML structural views | principal classes, inheritance, ownership and relevant multiplicities



The two structure pages form one model. Class compartments and operation contracts follow on pages 3-6.

Green = existing Stage 2 type; amber = existing type changed; purple = new Stage 3 type. Colours compare the implemented stages.

Hollow triangle points to a superclass; filled diamond means ownership; hollow diamond means shared aggregation.

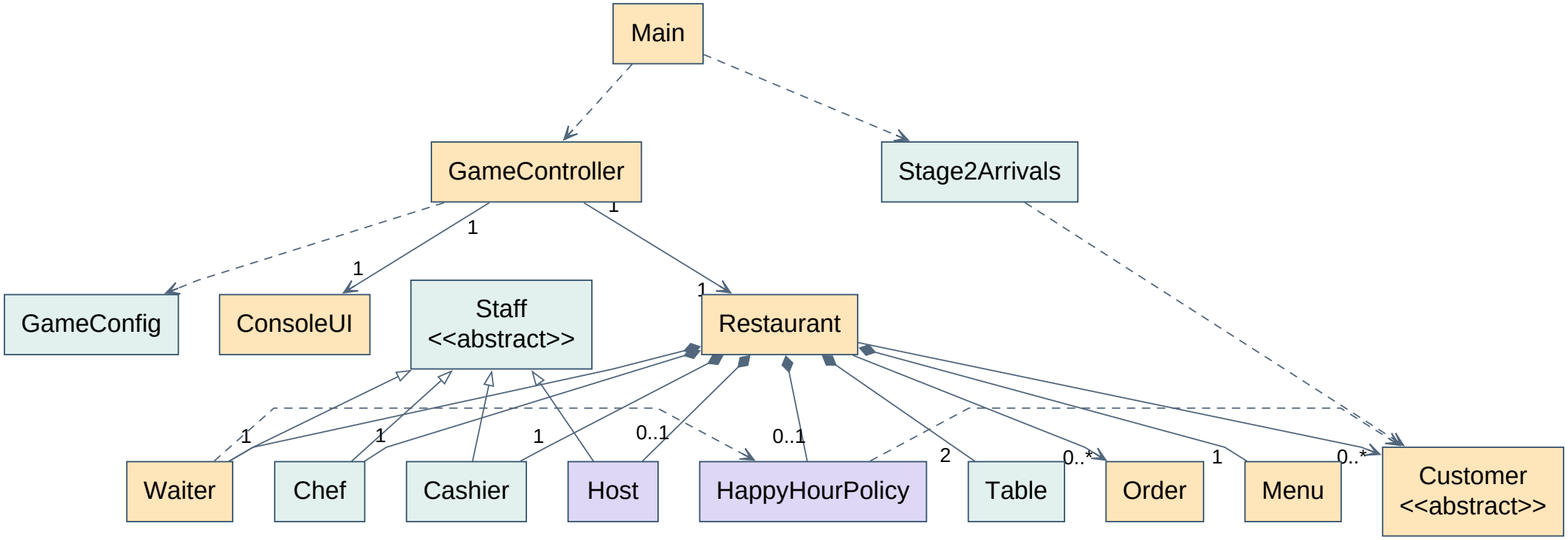
Menu owns 3 base items, plus one combo in Stage 3. Combo references two of those same items; it does not own them.

Solid lines are associations; dashed arrows are dependencies. Multiplicities describe objects, not fixed arrival totals.

RESTAURANT RUSH / STAGE 3

Class relationships - setup, coordination and employees

UML structural views | principal classes, inheritance, ownership and relevant multiplicities



All Restaurant-owned/registered links have multiplicity 1 at Restaurant; the opposite multiplicity is shown at its target.

Host and HappyHourPolicy are each 0..1 across supported launch modes, exactly one each in Stage 3.

Customer registry and active orders are 0..* collections; the configured run still has exactly six arrivals.

Main wires the mode and customer factory. The controller coordinates; customer behaviour remains polymorphic.

Waiter calls HappyHourPolicy, which calls Customer.priceFor. Cashier reads the final bill; no payment-time discount.

RESTAURANT RUSH / STAGE 3

Customer and menu contracts - Part A

Visibility: + public, - private, # protected. Routine getters are omitted. Colours show changes since Stage 2.

<<abstract>> / Part A / MODIFIED

Customer

```
- id: int
- satisfaction: int
- state: CustomerState
- table: Table
- order: Order
- servedTurn: int
- readyToPayTurn: int
```

```
+ waitingLoss(): int {abstract}
+ priceFor(item: MenuItem): int
+ getTypeLabel(): String
+ loseWaitingSatisfaction(): void
+ restoreWaitingSatisfaction(amount: int): void
+ chooseMeal(menu: Menu): MenuItem
+ seat(table: Table): void
+ placeOrder(order: Order): void
+ markServed(turn: int): void
+ finishEating(turn: int): void
+ canPay(turn: int): boolean
+ markPaid(): void
+ leave(): void
```

Encapsulation: changes to seating, service and satisfaction pass through state-checked methods. Waiting recovery applies to queued or seated customers and stops at 100.

<<enumeration>> / Part A / EXISTING

CustomerState

```
QUEUED, SEATED, SERVED, READY_TO_PAY, PAID, LEFT
```

<<class>> / Part A / EXISTING

RegularCustomer

```
+ waitingLoss(): int
```

<<class>> / Part A / EXISTING

VIPCustomer

```
+ getTypeLabel(): String
+ waitingLoss(): int
+ priceFor(item: MenuItem): int
```

<<class>> / Part A / EXISTING

CriticCustomer

```
+ getTypeLabel(): String
+ waitingLoss(): int
+ chooseMeal(menu: Menu): MenuItem
```

Polymorphism: the Waiter asks every customer to chooseMeal. The Critic implementation always returns Pasta.

<<class>> / Part A / MODIFIED

Menu

```
- items: List<MenuItem>
- combo: ComboMeal
```

```
+ Menu()
+ Menu(includeCombo: boolean)
+ hasCombo(): boolean
+ getCombo(): ComboMeal
+ getItems(): List<MenuItem>
+ find(name: String): MenuItem
```

The Stage 3 menu stores one ComboMeal and includes that same instance in its list of four dishes.

<<class>> / Part A / NEW

ComboMeal

```
- first: MenuItem
- second: MenuItem
```

```
+ ComboMeal(name: String, first: MenuItem,
second: MenuItem)
+ getFirst(): MenuItem
+ getSecond(): MenuItem
- comboPrice(first: MenuItem, second: MenuItem):
int {static}
```

The combo references two distinct basic items. Its base price is 90% of their total, rounded to the nearest cent. Preparation units add.

RESTAURANT RUSH / STAGE 3

Locked bills and Happy Hour - Part B

All monetary values are integer cents. Discounts never mutate menu data or bills that were already captured.

<<class>> / Part B / MODIFIED

Order

```
- customer: Customer
- item: MenuItem
- lockedPriceCents: int
- unitsDone: int
- status: OrderStatus
- readyTurn: int
```

```
+ Order(customer: Customer, item: MenuItem)
+ Order(customer: Customer, item: MenuItem,
lockedPriceCents: int)
+ getLockedPriceCents(): int
+ advance(turn: int): void
+ isServable(turn: int): boolean
+ markServed(): void
+ markPaid(): void
+ cancel(): void
```

The order retains its final price in cents. The cashier collects that amount even after Happy Hour expires.

<<enumeration>> / Part B / EXISTING

OrderStatus

```
PLACED, PREPARING, READY, SERVED, PAID,
CANCELLED
```

<<class>> / Part B / NEW

HappyHourPolicy

```
- DURATION: int {static}
- activationTurn: int
```

```
+ activate(turn: int): void
+ hasActivated(): boolean
+ isActive(turn: int): boolean
+ remainingTurns(turn: int): int
+ quote(customer: Customer, item: MenuItem,
turn: int): int
```

Pricing pipeline

- Combo base, if applicable: 90% of component sum.

- Customer quote: VIP 90%; others full item price.

- Active Happy Hour: 80% of customer quote.

- Order stores this final amount exactly once.

- Example: VIP Burger = 1500 -> 1350 -> 1080 cents.

<<class>> / Part A / EXISTING

MenuItem

```
- name: String
- priceCents: int
- prepUnits: int
```

```
+ getName(): String
+ getPriceCents(): int
+ getPrepUnits(): int
```

<<class>> / Part B / EXISTING

Table

```
- number: int
- occupant: Customer
```

```
+ isFree(): boolean
+ seat(customer: Customer): void
+ release(): void
```

<<class>> / Part D / EXISTING

Stage2Arrivals

```
+ create(id: int): Customer {static}
```

RESTAURANT RUSH / STAGE 3

Host and staff actions - Part C

The Host adds one independent seating action. The Waiter retains seating, ordering and serving.

<<abstract>> / Part C / EXISTING

Staff

```
- actedThisTurn: boolean
```

```
+ hasActed(): boolean
+ resetTurn(): void
+ skipTurn(): void
# requireAvailable(): void
# markActed(): void
```

<<class>> / Part C / NEW

Host

```
+ seat(customer: Customer, table: Table): void
```

<<class>> / Part C / MODIFIED

Waiter

```
+ seat(customer: Customer, table: Table): void
+ takeOrder(customer: Customer, menu: Menu):
Order
+ takeOrder(customer: Customer, menu: Menu,
policy: HappyHourPolicy, turn: int): Order
- chooseOrderItem(customer: Customer, menu:
Menu): MenuItem
+ serve(order: Order, turn: int): void
```

<<class>> / Part C / EXISTING

Chef

```
+ advance(order: Order, turn: int): void
```

<<class>> / Part C / EXISTING

Cashier

```
- revenueCents: int
- paidSatisfactions: List<Integer>
```

```
+ getRevenueCents(): int
+ getPaidCount(): int
+ getAverageSatisfaction(): double
+ collectPayment(customer: Customer, turn: int):
int
```

Integration invariants

- Host acts before Waiter. A customer seated by Host may order during that same turn.

- An invalid action leaves tables, customers, orders and task availability unchanged.

- Controller removes from queue only after seat succeeds; staff do not own the queue.

- Chef treats the combo as an ordinary MenuItem with summed preparation units.

- No type checks for VIP/Critic in staff or controller; call customer methods.

- Dish completed by Chef this turn cannot be served until a later turn.

RESTAURANT RUSH / STAGE 3

Setup and turn integration - Part D

Existing constructor overloads, arrivalFactory and stageLabel are retained; Stage 1/2 command sequences remain valid.

<<class>> / Part D / MODIFIED

Restaurant

```
- tables: List<Table>
- waitingQueue: Queue<Customer>
- customers: List<Customer>
- activeOrders: List<Order>
- menu: Menu
- waiter: Waiter
- chef: Chef
- cashier: Cashier
- stage3Enabled: boolean
- host: Host
- happyHour: HappyHourPolicy
- turn: int
- servedCount: int
- unhappyDepartures: int
```

```
+ Restaurant()
+ Restaurant(stage3Enabled: boolean)
+ isStage3Enabled(): boolean
+ getHost(): Host
+ getHappyHour(): HappyHourPolicy
+ findQueuedCustomer(id: int): Customer
+ findCustomer(id: int): Customer
+ findTable(number: int): Table
+ getOrder(customerId: int): Order
+ nextTurn(): void
+ addArrival(customer: Customer): void
+ removeFromQueue(customer: Customer): void
+ addOrder(order: Order): void
+ removeOrder(order: Order): void
+ handleDeparture(customer: Customer): void
```

Composition: each game constructs its own tables, menu and employees. The controller routes actions through those objects.

<<control>> / Part D / MODIFIED

GameController

```
- HAPPY_HOUR_RECOVERY: int {static}
- restaurant: Restaurant
- ui: ConsoleUI
- arrivalFactory: IntFunction<Customer>
```

```
+ GameController(restaurant: Restaurant, ui:
ConsoleUI)
+ GameController(restaurant: Restaurant, ui:
ConsoleUI, arrivalFactory:
IntFunction<Customer>)
+ run(): void
- processTurn(): void
- processArrivals(): void
- processHappyHourChoice(): void
- restoreWaitingCustomers(): void
- processHostChoice(): void
- processWaiterChoice(): void
- processChefChoice(): void
- processEatingAndPayments(): void
- processWaitingAndAbandonment(): void
- resetStaffActions(): void
- checkVictory(): boolean
```

<<class>> / Part D / MODIFIED

Main

```
+ main(args: String[]): void {static}
```

<<boundary>> / Part D / MODIFIED

ConsoleUI

```
- input: Scanner
- output: PrintStream
- stageLabel: String
```

```
+ ConsoleUI(input: InputStream, output:
PrintStream)
+ ConsoleUI(input: InputStream, output:
PrintStream, stageLabel: String)
+ showWelcome(): void
+ readCommand(): String
+ readCommand(prompt: String): String
+ showTurnState(restaurant: Restaurant): void
+ showHappyHour(policy: HappyHourPolicy, turn:
int): void
+ showOrderTaken(order: Order): void
+ showPayment(customer: Customer, cents: int):
void
+ showTurnEnd(restaurant: Restaurant): void
+ showResult(restaurant: Restaurant, victory:
boolean): void
```

<<class>> / Part D / EXISTING

GameConfig

```
+ MAX_TURNS: int {static}
+ TABLE_COUNT: int {static}
+ MIN_PAID_CUSTOMERS: int {static}
+ MIN_REVENUE_CENTS: int {static}
+ MIN_AVERAGE_SATISFACTION: int {static}
+ ARRIVAL_TURNS: List<Integer> {static}
```